

Week3:

Problem1:

using System;

using System.Collections.Generic;

class Program

```
{
    static void Main(string[] args)
    {
        // List to store 5 subject marks
        List<int> marks = new List<int>();
        marks.Add(85);
        marks.Add(72);
        marks.Add(60);
        marks.Add(90);
        marks.Add(75);

        Console.WriteLine("---- Student Result Analyzer ----");

        // Calling overloaded method directly (List version)
        Console.WriteLine("Average using List: " + CalculateAverage(marks));
        PrintGrade(CalculateAverage(marks));

        Console.WriteLine();

        // Calling overloaded method directly (5 parameters version)
        Console.WriteLine("Average using Parameters: " + CalculateAverage(85, 72, 60, 90, 75));
        PrintGrade(CalculateAverage(85, 72, 60, 90, 75));
    }

    // Method 1: Calculate Average using List<int>
    static double CalculateAverage(List<int> marks)
    {
        int sum = 0;

        foreach (int mark in marks)
        {
            sum += mark;
        }

        return (double)sum / marks.Count;
    }

    // Method Overloading: Calculate Average using 5 integers
    static double CalculateAverage(int m1, int m2, int m3, int m4, int m5)
    {
        int sum = m1 + m2 + m3 + m4 + m5;
        return (double)sum / 5;
    }

    // Method to Print Grade
    static void PrintGrade(double average)
    {
        if (average > 80)
        {
            Console.WriteLine("Grade: A");
        }
        else if (average > 60)
        {
            Console.WriteLine("Grade: B");
        }
        else if (average > 40)
        {

```

```

        Console.WriteLine("Grade: C");
    }
    else
    {
        Console.WriteLine("Grade: Fail");
    }
}
}

```

Problem2: Custom Calculator

using System;

class Program

```

{
    static void Main(string[] args)
    {
        Console.WriteLine("---- Custom Calculator ----");

        Console.WriteLine("Addition: " + Calculate(10, 5, "add"));
        Console.WriteLine("Subtraction: " + Calculate(10, 5, "subtract"));
        Console.WriteLine("Multiplication: " + Calculate(10, 5, "multiply"));
        Console.WriteLine("Division: " + Calculate(10, 5, "divide"));

        // Divide by zero case
        Console.WriteLine("Divide by Zero: " + Calculate(10, 0, "divide"));
    }

    // Method with parameters and return type
    static double Calculate(double a, double b, string operation)
    {
        return operation switch
        {
            "add" => a + b,
            "subtract" => a - b,
            "multiply" => a * b,
            "divide" => b == 0 ? 0 : a / b,
            _ => 0
        };
    }
}

```

Problem3: power calculator using recursive method

using System;

class Program

```

{
    static void Main(string[] args)
    {
        Console.WriteLine("---- power calculator (recursion) ----");

        Console.WriteLine("power(2,3): " + Power(2, 3));
        Console.WriteLine("power(5,2): " + Power(5, 2));
    }

    // recursive method to calculate power
    static int Power(int baseNum, int exponent)
    {
        // base case
        if (exponent == 0)
        {
            return 1;
        }
    }
}

```

```

        // recursive call
        return baseNum * Power(baseNum, exponent - 1);
    }
}
*****

```

Problem4: Dictionary-Based Product Inventory

```

using System;
using System.Collections.Generic;

```

class Program

```

{
    static void Main(string[] args)
    {
        Dictionary<string, double> products = new Dictionary<string, double>();

        AddProduct(products, "laptop", 55000);
        AddProduct(products, "mobile", 20000);

        DisplayAllProducts(products);

        RemoveProduct(products, "mobile");

        DisplayAllProducts(products);
    }

    // add product
    static void AddProduct(Dictionary<string, double> products, string name, double price)
    {
        products[name] = price;
    }

    // remove product
    static void RemoveProduct(Dictionary<string, double> products, string name)
    {
        products.Remove(name);
    }

    // display all products
    static void DisplayAllProducts(Dictionary<string, double> products)
    {
        foreach (var item in products)
        {
            Console.WriteLine(item.Key + " : " + item.Value);
        }
    }

    // find product price
    static double FindProductPrice(Dictionary<string, double> products, string name)
    {
        if (products.ContainsKey(name))
        {
            return products[name];
        }
    }
}

```

```

return 0;
}
}

```

Problem5: Build a Student Management Class

```

using System;
using System.Collections.Generic;

```

class Program

```

{
    static void Main(string[] args)
    {
        List<Student> students = new List<Student>();

        Student s1 = new Student();
        s1.Id = 1;
        s1.Name = "Kunal";
        s1.Marks = 90;

        Student s2 = new Student();
        s2.Id = 2;
        s2.Name = "Rahul";
        s2.Marks = 75;

        Student s3 = new Student();
        s3.Id = 3;
        s3.Name = "Amit";
        s3.Marks = 40;

        students.Add(s1);
        students.Add(s2);
        students.Add(s3);

        foreach (var s in students)
        {
            s.Display();
        }
    }
}

```

```

class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Marks { get; set; }

    public string CalculateGrade()
    {
        if (Marks >= 85)
        {
            return "A";
        }
        else if (Marks >= 70)
        {
            return "B";
        }
        else if (Marks >= 50)
        {
            return "C";
        }
        else
        {
            return "Fail";
        }
    }

    public void Display()
    {
        Console.WriteLine("Id: " + Id +
            ", Name: " + Name +
            ", Marks: " + Marks +
            ", Grade: " + CalculateGrade());
    }
}

```

```

    }
}
*****

```

Problem6: Product Class with Constructor Overloading
using System;

```

class Program
{
    static void Main(string[] args)
    {
        // using default constructor
        Product p1 = new Product();

        // using parameterized constructor
        Product p2 = new Product("mobile", 20000, 3);

        Console.WriteLine("product 1 total: " + p1.GetTotalPrice());
        Console.WriteLine("product 2 total: " + p2.GetTotalPrice());
    }
}

```

```

class Product
{
    // fields
    public string Name;
    public double Price;
    public int Quantity;

    // default constructor
    public Product()
    {
        Name = "default product";
        Price = 1000;
        Quantity = 1;
    }

    // parameterized constructor
    public Product(string name, double price, int quantity)
    {
        Name = name;
        Price = price;
        Quantity = quantity;
    }

    // method
    public double GetTotalPrice()
    {
        return Price * Quantity;
    }
}

```

Problem7: Encapsulation with Validation
using System;

```

class Program
{
    static void Main(string[] args)
    {
        Employee emp = new Employee();

        emp.SetSalary(50000);
        emp.SetAge(30);
    }
}

```

```

        Console.WriteLine("salary: " + emp.GetSalary());
        Console.WriteLine("age: " + emp.GetAge());

        // invalid values
        emp.SetSalary(-1000);
        emp.SetAge(15);
    }
}

```

```

class Employee
{
    // private fields (data hiding)
    private double salary;
    private int age;

    // setter for salary with validation
    public void SetSalary(double amount)
    {
        if (amount > 0)
        {
            salary = amount;
        }
        else
        {
            Console.WriteLine("invalid salary");
        }
    }

    // getter for salary
    public double GetSalary()
    {
        return salary;
    }

    // setter for age with validation
    public void SetAge(int value)
    {
        if (value >= 18 && value <= 60)
        {
            age = value;
        }
        else
        {
            Console.WriteLine("invalid age");
        }
    }

    // getter for age
    public int GetAge()
    {
        return age;
    }
}

```

Problem8: Shape Area Calculator (Runtime Polymorphism)

using System;

using System.Collections.Generic;

```

class Program
{
    static void Main(string[] args)
    {
        // list of shapes
    }
}

```

```

List<Shape> shapes = new List<Shape>();

shapes.Add(new Circle(5));
shapes.Add(new Rectangle(4, 6));
shapes.Add(new Triangle(3, 4));

Console.WriteLine("---- calculating areas ----");

foreach (Shape s in shapes)
{
    Console.WriteLine("area: " + s.CalculateArea());
}
}

// parent abstract class
abstract class Shape
{
    public abstract double CalculateArea();
}

// child class circle
class Circle : Shape
{
    private double radius;

    public Circle(double r)
    {
        radius = r;
    }

    public override double CalculateArea()
    {
        return 3.14 * radius * radius;
    }
}

// child class rectangle
class Rectangle : Shape
{
    private double length;
    private double width;

    public Rectangle(double l, double w)
    {
        length = l;
        width = w;
    }

    public override double CalculateArea()
    {
        return length * width;
    }
}

// child class triangle
class Triangle : Shape
{
    private double baseValue;
    private double height;

    public Triangle(double b, double h)
    {

```

```

        baseValue = b;
        height = h;
    }

    public override double CalculateArea()
    {
        return 0.5 * baseValue * height;
    }
}
*****
Problem8:Shape Area Calculator (Runtime Polymorphism)
using System;
using System.Collections.Generic;

class Program
{
    static void Main(string[] args)
    {
        // list of shapes
        List<Shape> shapes = new List<Shape>();

        shapes.Add(new Circle(5));
        shapes.Add(new Rectangle(4, 6));
        shapes.Add(new Triangle(3, 4));

        Console.WriteLine("---- calculating areas ----");

        foreach (Shape s in shapes)
        {
            Console.WriteLine("area: " + s.CalculateArea());
        }
    }
}

// parent abstract class
abstract class Shape
{
    public abstract double CalculateArea();
}

// child class circle
class Circle : Shape
{
    private double radius;

    public Circle(double r)
    {
        radius = r;
    }

    public override double CalculateArea()
    {
        return 3.14 * radius * radius;
    }
}

// child class rectangle
class Rectangle : Shape
{
    private double length;
    private double width;

    public Rectangle(double l, double w)

```



```

    {
        length = l;
        width = w;
    }

    public override double CalculateArea()
    {
        return length * width;
    }
}

```

```

// child class triangle
class Triangle : Shape
{
    private double baseValue;
    private double height;

    public Triangle(double b, double h)
    {
        baseValue = b;
        height = h;
    }

    public override double CalculateArea()
    {
        return 0.5 * baseValue * height;
    }
}

```

Problem9: Bank Account System (Method Overriding)
 using System;
 using System.Collections.Generic;

```

class Program
{
    static void Main(string[] args)
    {
        List<Account> accounts = new List<Account>();

        accounts.Add(new SavingsAccount(10000));
        accounts.Add(new CurrentAccount(10000));

        Console.WriteLine("---- interest calculation ----");

        foreach (Account acc in accounts)
        {
            Console.WriteLine("interest: " + acc.CalculateInterest());
        }
    }
}

```

```

// parent class
class Account
{
    protected double balance;

    public Account(double b)
    {
        balance = b;
    }

    public virtual double CalculateInterest()
    {

```

```

        return 0;
    }
}

// child class savings
class SavingsAccount : Account
{
    public SavingsAccount(double b) : base(b) { }

    public override double CalculateInterest()
    {
        return balance * 0.05; // 5% interest
    }
}

// child class current
class CurrentAccount : Account
{
    public CurrentAccount(double b) : base(b) { }

    public override double CalculateInterest()
    {
        return balance * 0.02; // 2% interest
    }
}
*****

Problme10: Vehicle Service System (Polymorphism + Encapsulation)
using System;
using System.Collections.Generic;

class Program
{
    static void Main(string[] args)
    {
        List<Vehicle> vehicles = new List<Vehicle>();

        vehicles.Add(new Car());
        vehicles.Add(new Bike());
        vehicles.Add(new Truck());

        double totalCost = 0;

        Console.WriteLine("---- service details ----");

        foreach (Vehicle v in vehicles)
        {
            v.StartEngine();
            double cost = v.ServiceCost();
            Console.WriteLine("service cost: " + cost);
            totalCost += cost;
        }

        Console.WriteLine("total service cost: " + totalCost);
    }
}

// parent class
class Vehicle
{
    // encapsulated field
    protected double baseCost = 1000;

    public void StartEngine()

```

```

    {
        Console.WriteLine("engine started");
    }

    public virtual double ServiceCost()
    {
        return baseCost;
    }
}

// child class car
class Car : Vehicle
{
    public override double ServiceCost()
    {
        return baseCost + 2000;
    }
}

// child class bike
class Bike : Vehicle
{
    public override double ServiceCost()
    {
        return baseCost + 500;
    }
}

// child class truck
class Truck : Vehicle
{
    public override double ServiceCost()
    {
        return baseCost + 5000;
    }
}
*****
*****

```

Problem1: Even-Odd Transformation Pipeline

```

using System;
using System.Linq;

```

```

class Program
{
    static void Main()
    {
        int[] numbers = { 3, 6, 9, 12, 6, 3, 15, 18, 2, 5 };

        var result = numbers
            .Distinct()           // remove duplicates
            .Where(n => n % 3 == 0) // divisible by 3
            .Select(n => n * n)     // square
            .OrderByDescending(n => n); // sort descending

        Console.WriteLine("result:");

        foreach (var n in result)
        {
            Console.WriteLine(n);
        }
    }
}
*****

```

Problem2: Custom Stack with History Tracking
using System;
using System.Collections.Generic;

```
// custom exception
class EmptyStackException : Exception
{
    public EmptyStackException(string message) : base(message) { }
}

class CustomStack
{
    // main stack
    private Stack<int> stack = new Stack<int>();

    // history stack
    private Stack<string> history = new Stack<string>();

    public void Push(int value)
    {
        stack.Push(value);
        history.Push("push");
    }

    public int Pop()
    {
        if (stack.Count == 0)
            throw new EmptyStackException("stack is empty");

        int value = stack.Pop();
        history.Push("pop");
        return value;
    }

    public int Peek()
    {
        if (stack.Count == 0)
            throw new EmptyStackException("stack is empty");

        return stack.Peek();
    }

    public void UndoLastOperation()
    {
        if (history.Count == 0)
        {
            Console.WriteLine("no operation to undo");
            return;
        }

        string last = history.Pop();

        if (last == "push")
        {
            stack.Pop();
            Console.WriteLine("undo push");
        }
        else if (last == "pop")
        {
            Console.WriteLine("cannot undo pop (value lost)");
        }
    }
}
```

```

class Program
{
    static void Main()
    {
        CustomStack cs = new CustomStack();

        cs.Push(10);
        cs.Push(20);

        Console.WriteLine("peek: " + cs.Peek());

        Console.WriteLine("pop: " + cs.Pop());

        cs.UndoLastOperation();
    }
}
*****

```

Problem3: Task Scheduler with Queue + Priority

```

using System;
using System.Collections.Generic;
using System.Linq;

```

```

class TaskItem
{
    public string Name;
    public int Priority; // 1 = high, 2 = medium, 3 = low

    public TaskItem(string name, int priority)
    {
        Name = name;
        Priority = priority;
    }
}

```

```

class Program
{
    static void Main()
    {
        List<TaskItem> tasks = new List<TaskItem>();

        tasks.Add(new TaskItem("Task1", 2));
        tasks.Add(new TaskItem("Task2", 1));
        tasks.Add(new TaskItem("Task3", 3));
        tasks.Add(new TaskItem("Task4", 1));

        Console.WriteLine("---- processing tasks ----");

        var sortedTasks = tasks
            .OrderBy(t => t.Priority); // sort by priority (1 first)

        foreach (var task in sortedTasks)
        {
            Console.WriteLine("processing: " + task.Name +
                " priority: " + task.Priority);
        }
    }
}
*****

```

Problem4: Improved HashMap with Collision Detection

```

using System;
using System.Collections.Generic;

```

```

class SimpleHashMap
{
    private List<string>[] keys = new List<string>[10];
    private List<string>[] values = new List<string>[10];

    private int GetIndex(string key)
    {
        int hash = 0;

        foreach (char c in key)
            hash += c;

        return hash % keys.Length;
    }

    public void Add(string key, string value)
    {
        int index = GetIndex(key);

        if (keys[index] == null)
        {
            keys[index] = new List<string>();
            values[index] = new List<string>();
        }

        if (keys[index].Count > 0)
            Console.WriteLine("collision detected at index " + index);

        keys[index].Add(key);
        values[index].Add(value);
    }

    public string Get(string key)
    {
        int index = GetIndex(key);

        if (keys[index] != null)
        {
            for (int i = 0; i < keys[index].Count; i++)
            {
                if (keys[index][i] == key)
                    return values[index][i];
            }
        }

        return "key not found";
    }

    public void Remove(string key)
    {
        int index = GetIndex(key);

        if (keys[index] != null)
        {
            for (int i = 0; i < keys[index].Count; i++)
            {
                if (keys[index][i] == key)
                {
                    keys[index].RemoveAt(i);
                    values[index].RemoveAt(i);
                    break;
                }
            }
        }
    }
}

```

```

    }
}

```

```

class Program
{
    static void Main()
    {
        SimpleHashMap map = new SimpleHashMap();

        map.Add("name", "kunal");
        map.Add("eman", "rahul"); // possible collision

        Console.WriteLine("Value: " + map.Get("name"));

        map.Remove("name");

        Console.WriteLine("After Remove: " + map.Get("name"));
    }
}
*****

```

Problem5: Binary Tree – Count & Height
using System;

```

record Person(string Name, int Age); // immutable

```

```

struct Point
{
    public int X;
    public int Y;
}

```

```

class Node
{
    public int Value;
    public Node? Left;
    public Node? Right;

    public Node(int value)
    {
        Value = value;
    }
}

```

```

class Program
{
    static void Main()
    {
        Node root = new Node(10);
        root.Left = new Node(5);
        root.Right = new Node(20);
        root.Left.Left = new Node(2);
        root.Left.Right = new Node(7);

        Console.WriteLine("Inorder Traversal:");
        InOrder(root);

        Console.WriteLine("Total Nodes: " + CountNodes(root));
        Console.WriteLine("Height: " + GetHeight(root));
        Console.WriteLine("Min Value: " + FindMin(root));
        Console.WriteLine("Max Value: " + FindMax(root));

        Person p = new Person("Kunal", 26);
    }
}

```

```

        Console.WriteLine(p);

        Point pt;
        pt.X = 10;
        pt.Y = 20;
        Console.WriteLine($"Point: {pt.X}, {pt.Y}");
    }

    static void InOrder(Node? node)
    {
        if (node == null) return;

        InOrder(node.Left);
        Console.WriteLine(node.Value);
        InOrder(node.Right);
    }

    static int CountNodes(Node? node)
    {
        if (node == null) return 0;

        return 1 + CountNodes(node.Left) + CountNodes(node.Right);
    }

    static int GetHeight(Node? node)
    {
        if (node == null) return 0;

        int leftHeight = GetHeight(node.Left);
        int rightHeight = GetHeight(node.Right);

        return 1 + Math.Max(leftHeight, rightHeight);
    }

    static int FindMin(Node? node)
    {
        if (node?.Left == null)
            return node!.Value;

        return FindMin(node.Left);
    }

    static int FindMax(Node? node)
    {
        if (node?.Right == null)
            return node!.Value;

        return FindMax(node.Right);
    }
}

```

Problem6: Student File Processor with Error Logging

```

using System;
using System.IO;
using System.Linq;
using System.Collections.Generic;

class InvalidMarksException : Exception
{
    public InvalidMarksException(string message) : base(message) { }
}

```

class Student


```

{
    public string Name;
    public int Marks;

    public Student(string name, int marks)
    {
        Name = name;
        Marks = marks;
    }
}

class Program
{
    static void Main()
    {
        List<Student> students = new List<Student>();

        string[] lines = File.ReadAllLines("data.txt");

        foreach (string line in lines)
        {
            try
            {
                string[] parts = line.Split(',');

                string name = parts[0];
                string marksText = parts[1];

                if (!int.TryParse(marksText, out int marks))
                    throw new InvalidMarksException("Invalid marks for " + name);

                students.Add(new Student(name, marks));
            }
            catch (InvalidMarksException ex)
            {
                Console.WriteLine("Custom Error: " + ex.Message);

                // append error to file
                File.AppendAllText("errors.txt", ex.Message + Environment.NewLine);
            }
        }

        // LINQ average
        if (students.Count > 0)
        {
            double average = students.Average(s => s.Marks);
            Console.WriteLine("Average Marks: " + average);
        }

        Console.WriteLine("Validation Completed.");
    }
}

```

Problem7: Nullable Order Discount System
using System;

```

class InvalidDiscountException : Exception
{
    public InvalidDiscountException(string message) : base(message) { }
}

```

class Program

```

{
    static void Main()
    {
        double amount = 6000;
        double? discountPercentage = null; // try 50 to test exception

        try
        {
            // null default
            if (discountPercentage == null)
                discountPercentage = 10;

            // validation
            if (discountPercentage > 40)
                throw new InvalidDiscountException("Discount cannot exceed 40%");

            // pattern matching
            double extraDiscount = amount switch
            {
                > 5000 => 5,
                <= 5000 => 0
            };

            double totalDiscount = discountPercentage.Value + extraDiscount;

            double finalPrice = amount - (amount * totalDiscount / 100);

            Console.WriteLine("Final Price: " + finalPrice);
        }
        catch (InvalidDiscountException ex)
        {
            Console.WriteLine("Custom Error: " + ex.Message);
        }
    }
}
*****

```

Problem8: Employee Analytics using LINQ

```

using System;
using System.Linq;
using System.Collections.Generic;

```

```

class Employee
{
    public string Name;
    public string Department;
    public double Salary;

    public Employee(string name, string dept, double salary)
    {
        Name = name;
        Department = dept;
        Salary = salary;
    }
}

```

```

class Program
{
    static void Main()
    {
        List<Employee> employees = new List<Employee>
        {
            new Employee("Kunal", "IT", 50000),
            new Employee("Rahul", "HR", 40000),

```

```

        new Employee("Amit", "IT", 70000),
        new Employee("Neha", "HR", 45000),
        new Employee("Priya", "Finance", 60000)
    };

    // average salary
    double avgSalary = employees.Average(e => e.Salary);

    // salary > average
    var aboveAverage = employees
        .Where(e => e.Salary > avgSalary);

    Console.WriteLine("Employees with salary > average:");

    foreach (var e in aboveAverage)
        Console.WriteLine(e.Name + " - " + e.Salary);

    // group by department
    var result = employees
        .GroupBy(e => e.Department)
        .ToDictionary(
            g => g.Key,
            g => g.OrderByDescending(e => e.Salary).First()
        );

    Console.WriteLine("\nHighest salary employee per department:");

    foreach (var item in result)
        Console.WriteLine(item.Key + " -> " +
            item.Value.Name + " - " +
            item.Value.Salary);
    }
}
*****

```

Problem9: Tuple-Based Result Evaluation System

```

using System;
using System.IO;
using System.Linq;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        List<(string Name, int Marks)> students = new List<(string, int)>
        {
            ("Kunal", 95),
            ("Rahul", 70),
            ("Amit", 35),
            ("Neha", 80)
        };

        // evaluate and write to file
        foreach (var student in students)
        {
            var (grade, isPass) = Evaluate(student.Marks);

            string line = student.Name + "," +
                student.Marks + "," +
                grade + "," +
                isPass;

            File.AppendAllText("results.txt", line + Environment.NewLine);
        }
    }
}

```

```

    }

    Console.WriteLine("Passed Students:");

    // read file
    var lines = File.ReadAllLines("results.txt");

    foreach (var line in lines)
    {
        var parts = line.Split(',');

        string name = parts[0];
        bool isPass = bool.Parse(parts[3]);

        if (isPass)
            Console.WriteLine(name);
    }
}

static (string grade, bool isPass) Evaluate(int marks)
{
    string grade = marks switch
    {
        >= 90 => "A+",
        >= 75 => "A",
        >= 60 => "B",
        >= 40 => "C",
        _ => "Fail"
    };

    bool isPass = marks >= 40;

    return (grade, isPass);
}
}
*****

```

Problem10: Mini File-Based CRUD Student System

```

using System;
using System.IO;
using System.Linq;

record Student(int Id, string Name, int Marks);

class Program
{
    static string filePath = "students.txt";

    static void Main()
    {
        while (true)
        {
            Console.WriteLine("\n1. Add student");
            Console.WriteLine("2. View all");
            Console.WriteLine("3. Search by Id");
            Console.WriteLine("4. Delete student");
            Console.WriteLine("5. Exit");

            Console.Write("Choose: ");
            string choice = Console.ReadLine();

            try
            {
                switch (choice)

```

```

        {
            case "1":
                AddStudent();
                break;

            case "2":
                ViewAll();
                break;

            case "3":
                SearchById();
                break;

            case "4":
                DeleteStudent();
                break;

            case "5":
                return;

            default:
                Console.WriteLine("Invalid choice");
                break;
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine("Error: " + ex.Message);
    }
}

static void AddStudent()
{
    Console.Write("Enter Id: ");
    int id = int.Parse(Console.ReadLine());

    Console.Write("Enter Name: ");
    string name = Console.ReadLine();

    Console.Write("Enter Marks: ");
    int marks = int.Parse(Console.ReadLine());

    string data = $"{id},{name},{marks}";
    File.AppendAllText(filePath, data + Environment.NewLine);

    Console.WriteLine("Student added.");
}

static void ViewAll()
{
    if (!File.Exists(filePath))
    {
        Console.WriteLine("No records found.");
        return;
    }

    var lines = File.ReadAllLines(filePath);

    foreach (var line in lines)
        Console.WriteLine(line);
}

```

```

static void SearchById()
{
    Console.Write("Enter Id: ");
    int id = int.Parse(Console.ReadLine());

    var lines = File.ReadAllLines(filePath);

    var result = lines
        .Where(x => x.Split(',')[0] == id.ToString())
        .FirstOrDefault();

    if (result != null)
        Console.WriteLine("Found: " + result);
    else
        Console.WriteLine("Student not found.");
}

static void DeleteStudent()
{
    Console.Write("Enter Id: ");
    int id = int.Parse(Console.ReadLine());

    var lines = File.ReadAllLines(filePath);

    var updated = lines
        .Where(x => x.Split(',')[0] != id.ToString())
        .ToArray();

    File.WriteAllLines(filePath, updated);

    Console.WriteLine("Delete done.");
}
}
*****
Problem1:
using System;
using System.Collections.Generic;
using System.Threading;

class Program
{
    static void Main()
    {
        OrderService service = new OrderService();

        Thread t1 = new Thread(() => service.PlaceOrder(1));
        Thread t2 = new Thread(() => service.PlaceOrder(2));
        Thread t3 = new Thread(() => service.PlaceOrder(1)); // duplicate id test

        t1.Start();
        t2.Start();
        t3.Start();

        t1.Join();
        t2.Join();
        t3.Join();

        Console.WriteLine($"Total Orders: {service.OrderCount}");
    }
}

class OrderService
{

```

```

private readonly object _lock = new object();
private List<int> orders = new List<int>();

public int OrderCount
{
    get { return orders.Count; }
}

public void PlaceOrder(int orderId)
{
    lock (_lock)
    {
        if (!orders.Contains(orderId))
        {
            orders.Add(orderId);
            Console.WriteLine($"Order {orderId} placed by Thread {Thread.CurrentThread.ManagedThreadId}");
        }
        else
        {
            Console.WriteLine($"Duplicate Order {orderId} blocked by Thread {Thread.CurrentThread.ManagedThreadId}");
        }
    }
}
}
*****
using System;
using System.IO;
using System.Collections.Generic;
using System.Diagnostics;
using System.Threading.Tasks;

class Program
{
    static void Main()
    {
        List<string> files = new List<string>
        {
            "file1.txt",
            "file2.txt",
            "file3.txt",
            "file4.txt",
            "file5.txt"
        };

        // Sequential Version
        Stopwatch sw1 = Stopwatch.StartNew();
        int totalWords1 = 0;

        foreach (var file in files)
        {
            string text = File.ReadAllText(file);
            totalWords1 += text.Split(' ', StringSplitOptions.RemoveEmptyEntries).Length;
            Console.WriteLine($"Reading {file}");
        }

        sw1.Stop();
        Console.WriteLine($"Sequential Words: {totalWords1}");
        Console.WriteLine($"Sequential Time: {sw1.ElapsedMilliseconds} ms");

        // Parallel Version
        Stopwatch sw2 = Stopwatch.StartNew();
        int totalWords2 = 0;

```

```

Parallel.ForEach(files, file =>
{
    string text = File.ReadAllText(file);
    int count = text.Split(' ', StringSplitOptions.RemoveEmptyEntries).Length;

    lock (files)
    {
        totalWords2 += count;
    }

    Console.WriteLine($"Processing {file} Thread {Environment.CurrentManagedThreadId}");
});

sw2.Stop();
Console.WriteLine($"Parallel Words: {totalWords2}");
Console.WriteLine($"Parallel Time: {sw2.ElapsedMilliseconds} ms");
}
}
*****

```

Problem3:

using System;

using System.Threading.Tasks;

class Program

```

{
    static async Task Main()
    {
        try
        {
            Task<string> t1 = FetchUsersAsync();
            Task<string> t2 = FetchOrdersAsync();
            Task<string> t3 = FetchPaymentsAsync();

            string[] results = await Task.WhenAll(t1, t2, t3);

            foreach (var r in results)
            {
                Console.WriteLine(r);
            }
        }
        catch (Exception ex)
        {
            Console.WriteLine("Exception caught:");
            Console.WriteLine(ex.Message);
            Console.WriteLine(ex.StackTrace);
        }
    }

    static async Task<string> FetchUsersAsync()
    {
        await Task.Delay(2000);
        return "Users fetched";
    }

    static async Task<string> FetchOrdersAsync()
    {
        await Task.Delay(2000);
        return "Orders fetched";
    }

    static async Task<string> FetchPaymentsAsync()
    {

```



```

        await Task.Delay(2000);
        return "Payments fetched";
    }
}
*****

```

Problem4:

using System;

class Program

```

{
    static void Main()
    {
        try
        {
            Controller controller = new Controller();
            controller.ProcessRequest();
        }
        catch (Exception ex)
        {
            Console.WriteLine("Exception caught:");
            Console.WriteLine(ex.Message);
            Console.WriteLine(ex.StackTrace);
        }
    }
}

```

// ----- Controller -----

class Controller

```

{
    Service service = new Service();

    public void ProcessRequest()
    {
        Console.WriteLine("Enter Controller");
        service.HandleRequest();
        Console.WriteLine("Exit Controller");
    }
}

```

// ----- Service -----

class Service

```

{
    Repository repo = new Repository();

    public void HandleRequest()
    {
        Console.WriteLine("Enter Service");
        repo.GetData();
        Console.WriteLine("Exit Service");
    }
}

```

// ----- Repository -----

class Repository

```

{
    public void GetData()
    {
        Console.WriteLine("Enter Repository");

        throw new Exception("Database connection failed");

        Console.WriteLine("Exit Repository");
    }
}

```

```
}
*****
```

Problem5:

```
using System;
using System.Security.Cryptography;
using System.Text;
using Microsoft.Data.Sqlite;
```

class Program

```
{
    static void Main()
    {
        string connectionString = "Data Source=users.db";

        try
        {
            using var connection = new SqliteConnection(connectionString);
            connection.Open();

            // Create Table
            var create = connection.CreateCommand();
            create.CommandText = @"
CREATE TABLE IF NOT EXISTS Users(
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    Username TEXT,
    PasswordHash TEXT
);";
            create.ExecuteNonQuery();

            string username = "admin";
            string password = "1234";

            if (string.IsNullOrEmpty(username) || string.IsNullOrEmpty(password))
            {
                Console.WriteLine("Invalid input");
                return;
            }

            string hash = HashPassword(password);

            // Insert User (Parameterized)
            var insert = connection.CreateCommand();
            insert.CommandText = "INSERT INTO Users (Username, PasswordHash) VALUES ($u,$p)";
            insert.Parameters.AddWithValue("$u", username);
            insert.Parameters.AddWithValue("$p", hash);
            insert.ExecuteNonQuery();

            // Login Validation
            var login = connection.CreateCommand();
            login.CommandText = "SELECT PasswordHash FROM Users WHERE Username=$u";
            login.Parameters.AddWithValue("$u", username);

            using var reader = login.ExecuteReader();

            if (reader.Read())
            {
                string storedHash = reader.GetString(0);

                if (storedHash == HashPassword(password))
                    Console.WriteLine("Login Successful");
                else
                    Console.WriteLine("Invalid Password");
            }
        }
    }
}
```

```

    }
    catch (SqliteException ex)
    {
        Console.WriteLine("SQL Error:");
        Console.WriteLine(ex.Message);
    }
}

static string HashPassword(string password)
{
    using SHA256 sha = SHA256.Create();
    byte[] bytes = sha.ComputeHash(Encoding.UTF8.GetBytes(password));
    return Convert.ToBase64String(bytes);
}
}
*****

```

Problem6:

```

using System;
using Microsoft.Data.Sqlite;

```

class Program

```

{
    static void Main()
    {
        string connectionString = "Data Source=bank.db";

        using var connection = new SqliteConnection(connectionString);
        connection.Open();

        // Create Table
        var create = connection.CreateCommand();
        create.CommandText = @"
CREATE TABLE IF NOT EXISTS Accounts(
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    Name TEXT,
    Balance INTEGER
);";
        create.ExecuteNonQuery();

        // Start Transaction
        using var transaction = connection.BeginTransaction();

        try
        {
            // Withdraw from Account A
            var withdraw = connection.CreateCommand();
            withdraw.Transaction = transaction;
            withdraw.CommandText = "UPDATE Accounts SET Balance = Balance - 100 WHERE Name = 'A'";
            withdraw.ExecuteNonQuery();

            // Deposit to Account B
            var deposit = connection.CreateCommand();
            deposit.Transaction = transaction;
            deposit.CommandText = "UPDATE Accounts SET Balance = Balance + 100 WHERE Name = 'B'";
            deposit.ExecuteNonQuery();

            transaction.Commit();
            Console.WriteLine("Transfer Successful");
        }
        catch (Exception ex)
        {
            transaction.Rollback();
            Console.WriteLine("Transfer Failed");
        }
    }
}

```

```

        Console.WriteLine(ex.Message);
    }
}

```

Problem7:

```

using System;
using Microsoft.Data.Sqlite;

```

class Program

```

{
    static void Main()
    {
        string connectionString = "Data Source=users.db";

        using var connection = new SqliteConnection(connectionString);
        connection.Open();

        string input = "' OR 1=1 --";

        VulnerableLogin(connection, input);
        SecureLogin(connection, input);
    }

    // Vulnerable Method
    static void VulnerableLogin(SqliteConnection connection, string username)
    {
        var cmd = connection.CreateCommand();
        cmd.CommandText = "SELECT * FROM Users WHERE Username = '" + username + "'";

        using var reader = cmd.ExecuteReader();

        if (reader.Read())
            Console.WriteLine("Vulnerable Login Success");
        else
            Console.WriteLine("Vulnerable Login Failed");
    }

    // Secure Method
    static void SecureLogin(SqliteConnection connection, string username)
    {
        var cmd = connection.CreateCommand();
        cmd.CommandText = "SELECT * FROM Users WHERE Username = $username";
        cmd.Parameters.AddWithValue("$username", username);

        using var reader = cmd.ExecuteReader();

        if (reader.Read())
            Console.WriteLine("Secure Login Success");
        else
            Console.WriteLine("Secure Login Failed");
    }
}

```

Problem8:

```

using System;
using System.Collections.Generic;
using System.Threading.Tasks;

```

class Program

```

{
    static void Main()
    {

```

```

        Stock stock = new Stock();

        stock.Subscribe(new EmailSubscriber());
        stock.Subscribe(new SmsSubscriber());

        stock.SetPrice(100);

        Console.ReadLine();
    }
}

// ----- Observer -----
interface IObservable
{
    Task UpdateAsync(int price);
}

class EmailSubscriber : IObservable
{
    public async Task UpdateAsync(int price)
    {
        try
        {
            await Task.Run(() =>
            {
                Console.WriteLine($"Email: Stock price updated to {price}");
            });
        }
        catch (Exception ex)
        {
            Console.WriteLine(ex.Message);
        }
    }
}

class SmsSubscriber : IObservable
{
    public async Task UpdateAsync(int price)
    {
        try
        {
            await Task.Run(() =>
            {
                Console.WriteLine($"SMS: Stock price updated to {price}");
            });
        }
        catch (Exception ex)
        {
            Console.WriteLine(ex.Message);
        }
    }
}

// ----- Stock -----
class Stock
{
    private List<IObservable> observers = new List<IObservable>();

    public void Subscribe(IObservable observer)
    {
        observers.Add(observer);
    }
}

```

```
public void SetPrice(int price)
{
    foreach (var observer in observers)
    {
        _ = observer.UpdateAsync(price);
    }
}
```